



Evolvi la tua VPC

(senza downtime)

INTRO

Partiamo da Zero

Hai appena creato il tuo account AWS
Qual è la prima risorsa che crei?

Puoi già fare molto senza troppo setup:

- Lambda
- DynamoDB
- S3
- ...

Quando è ora di fare sul serio:

- EC2, ECS, ALB, RDS...
- Necessità di uno stack di rete strutturato

È ora di creare la tua prima **VPC**



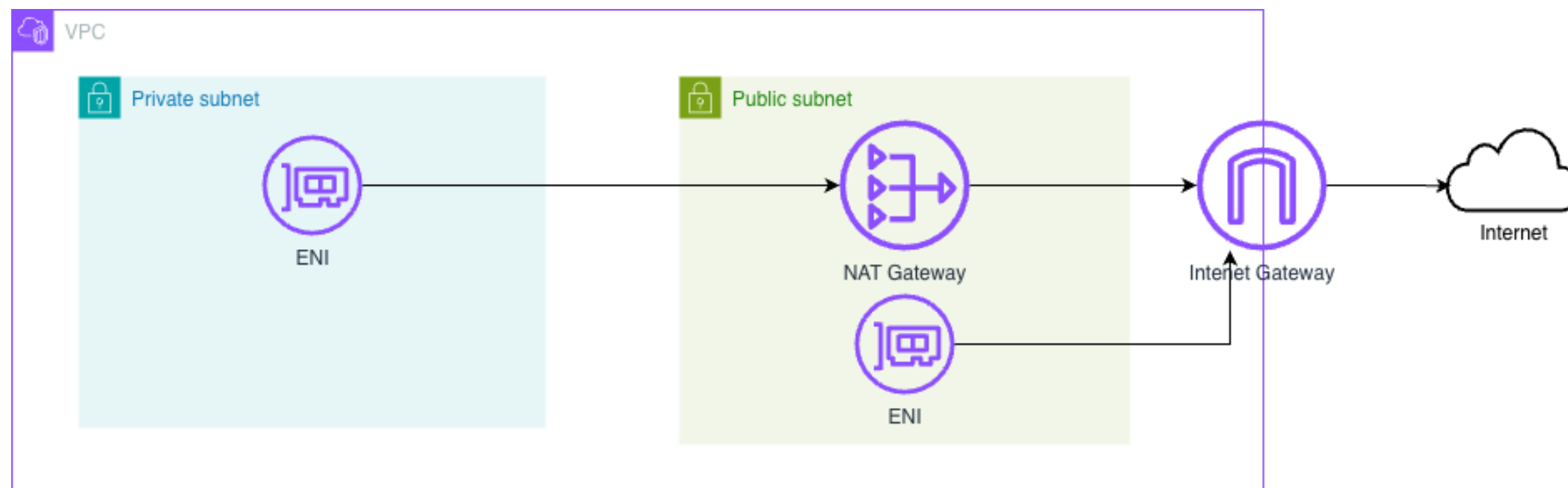
INTRO

VPC

Una **VPC (Virtual Private Cloud)** è una rete virtuale logicamente isolata nel cloud dove puoi eseguire risorse (come server e database) con controllo su indirizzi IP, subnet, routing e sicurezza.

Sottorete pubblica: Una subnet con accesso diretto a Internet tramite un Internet Gateway.

Sottorete privata: Una subnet non accessibile direttamente da internet. Le risorse al suo interno e necessitano di un NAT Gateway per comunicare verso l'esterno



INTRO

E ora?

Crei la tua prima VPC da console, cosa potrebbe mai andare storto?

In fondo, hai studiato i CIDR, sai fare un diagramma rete e tutto funziona, no? Al massimo la puoi sempre rifare

Fast-forward 10 anni 

- La tua infrastruttura è ancora su
- Serve milioni di chiamate e TB di dati ogni mese
- I tuoi servizi scalano frequentemente

I temi di **High Availability**, **Scalabilità** e **Sicurezza** non sono più così astratti

La tua VPC è sempre lì sotto, ma comincia a diventare un problema





EVOLVI LA TUA VPC

INDICE

- **IL NOSTRO CASO**
- **PLANNING**
 - 1. Ristrutturazione CIDR**
 - 2. HA**
 - 3. IPv6**
- **NEL PRATICO**
 1. Aggiornare la VPC
 2. Migrare le risorse
- **KEY TAKEWAYS**



IL NOSTRO CASO

Una storia che parte da molto
lontano

IL NOSTRO CASO

1. 2011

THRON comincia ad usare AWS e crea la prima sua prima VPC

Molte delle tecnologie che oggi diamo per scontato non esistevano:

- Lambda
- Docker
- IaC

La prima **VPC** è creata a manualmente e si include connettività a **server on prem**

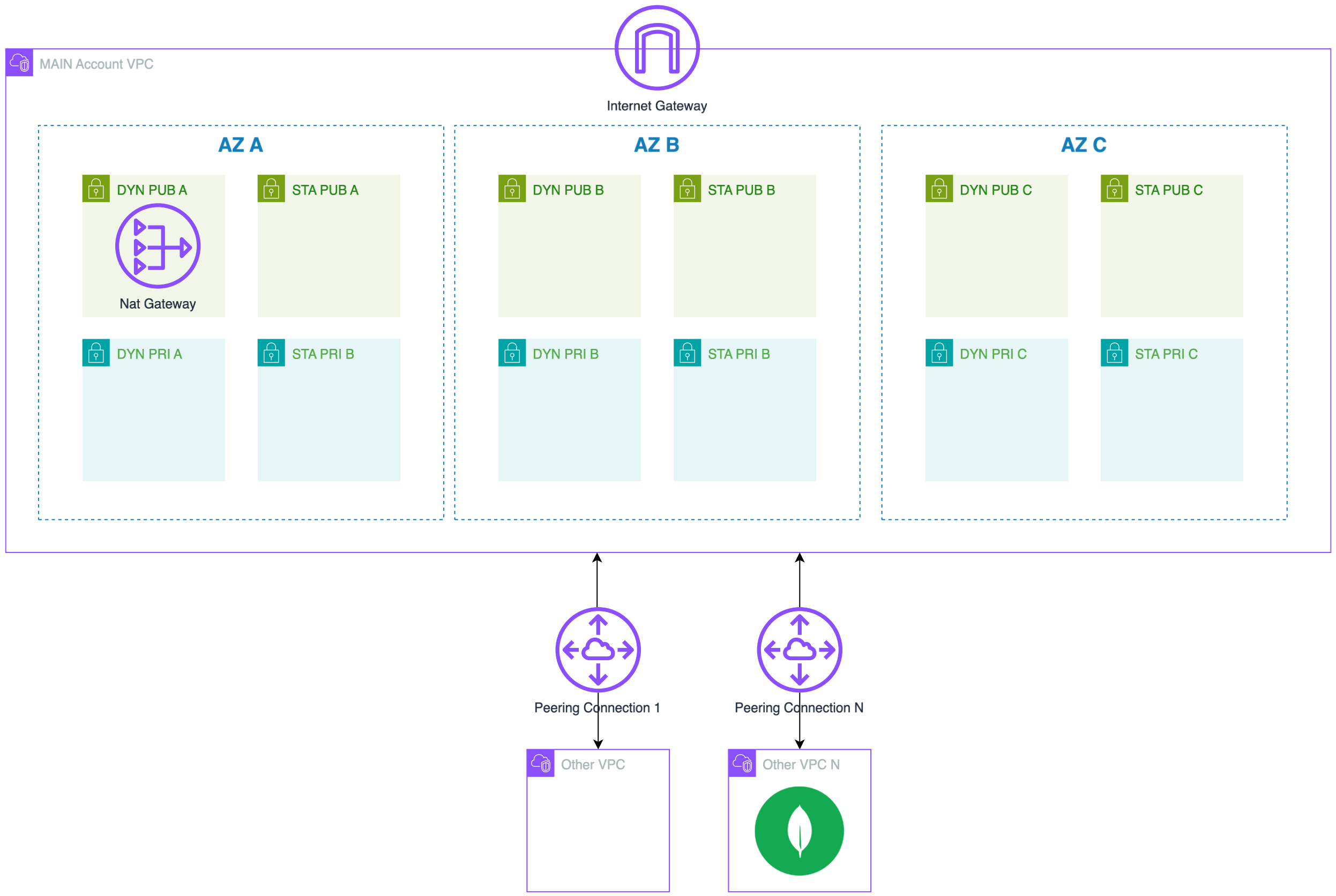
2025

Il 100% dei workload di **THRON** girano nel cloud, in gran parte su **AWS**.

Tutta l'infrastruttura è replicata su 3 ambienti

Non ci sono più connessioni con server fisici, ma:

- Peering con VPC secondarie
- Peering con servizi come MongoDB Atlas



IL NOSTRO CASO

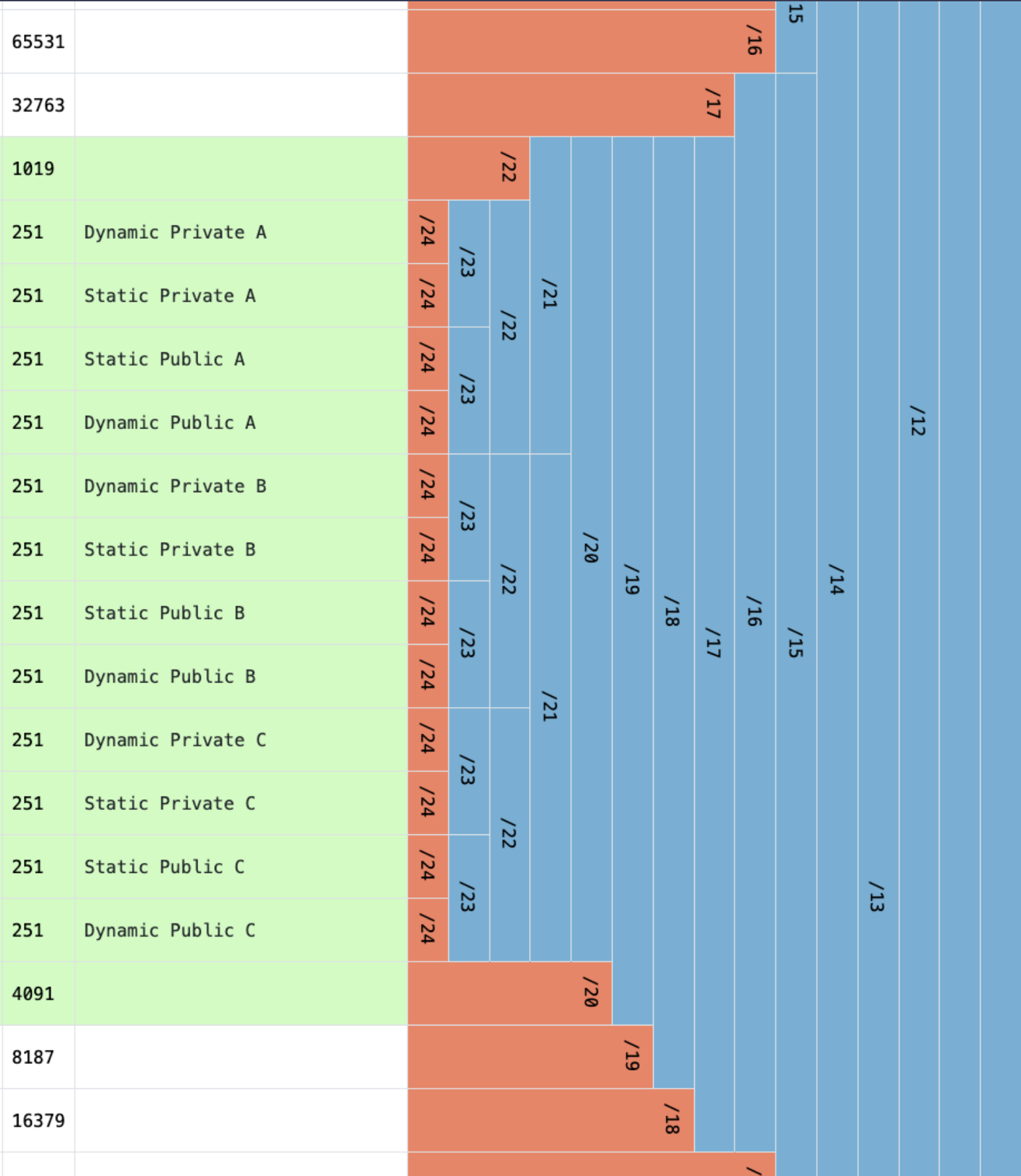
1. Frammentazione Sottoreti

- La VPC è un blocco /19 diviso 12 in /24:
- In origine, RouteTables separate per routing on-prem
 - Alcune sottoreti **over/under utilizzate**
 - Sottoreti **piccole**, 251 indirizzi ciascuna

Nessun problema per tecnologie un po' più classiche. In **ECS** l'ip dell'**host** è condiviso da più servizi

- Da qualche anno, massiccia introduzione di tecnologie serverless:
- **Lambda**: 3 IP per ogni funzione (1 per subnet)
 - **Fargate**: 1 IP per task

TRADOTTO: STAVAMO FINENDO GLI INDIRIZZI IP



IL NOSTRO CASO

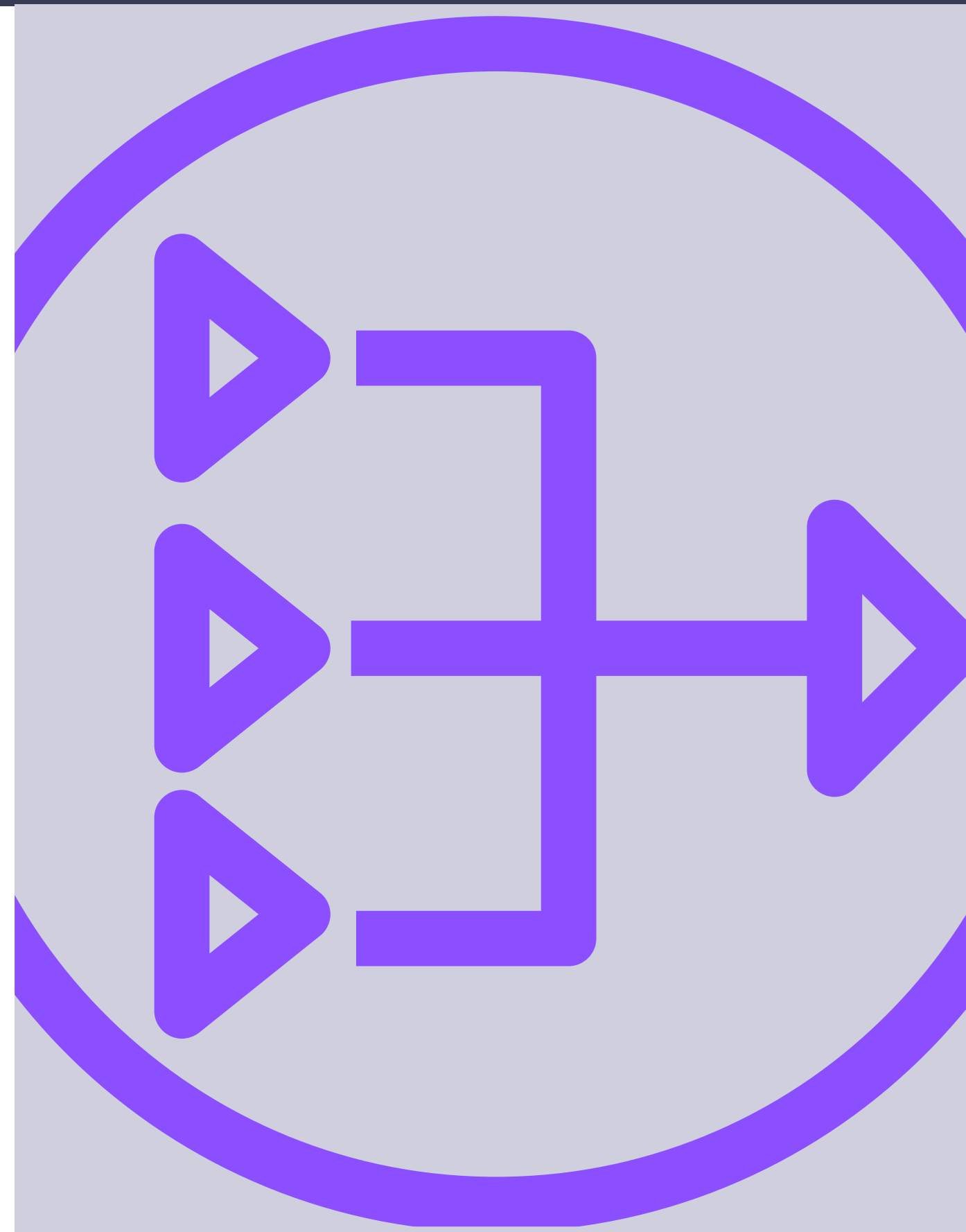
1. NAT in singola AZ

Sottoreti private instradate attraverso un **unico** NAT nell'AZ A.

Funzionava bene perché:

- Maggior parte dei servizi usava S3 e DynamoDB Gateway endpoints
- Minime dipendenze da Internet esterno
- In caso di failure AZ A: perdita accesso Internet ma servizi operativi
- Alcuni servizi su cluster ECS in subnet pubbliche per aggirare il limite

Con la maturazione dell'infrastruttura e l'adozione serverless, necessario passare a una soluzione meno vincolata.



IL NOSTRO CASO

1. Evolvere, non distruggere

Creiamo una nuova VPC, facciamo qualche trick con peering, e poi migriamo tutto, no? Non proprio:

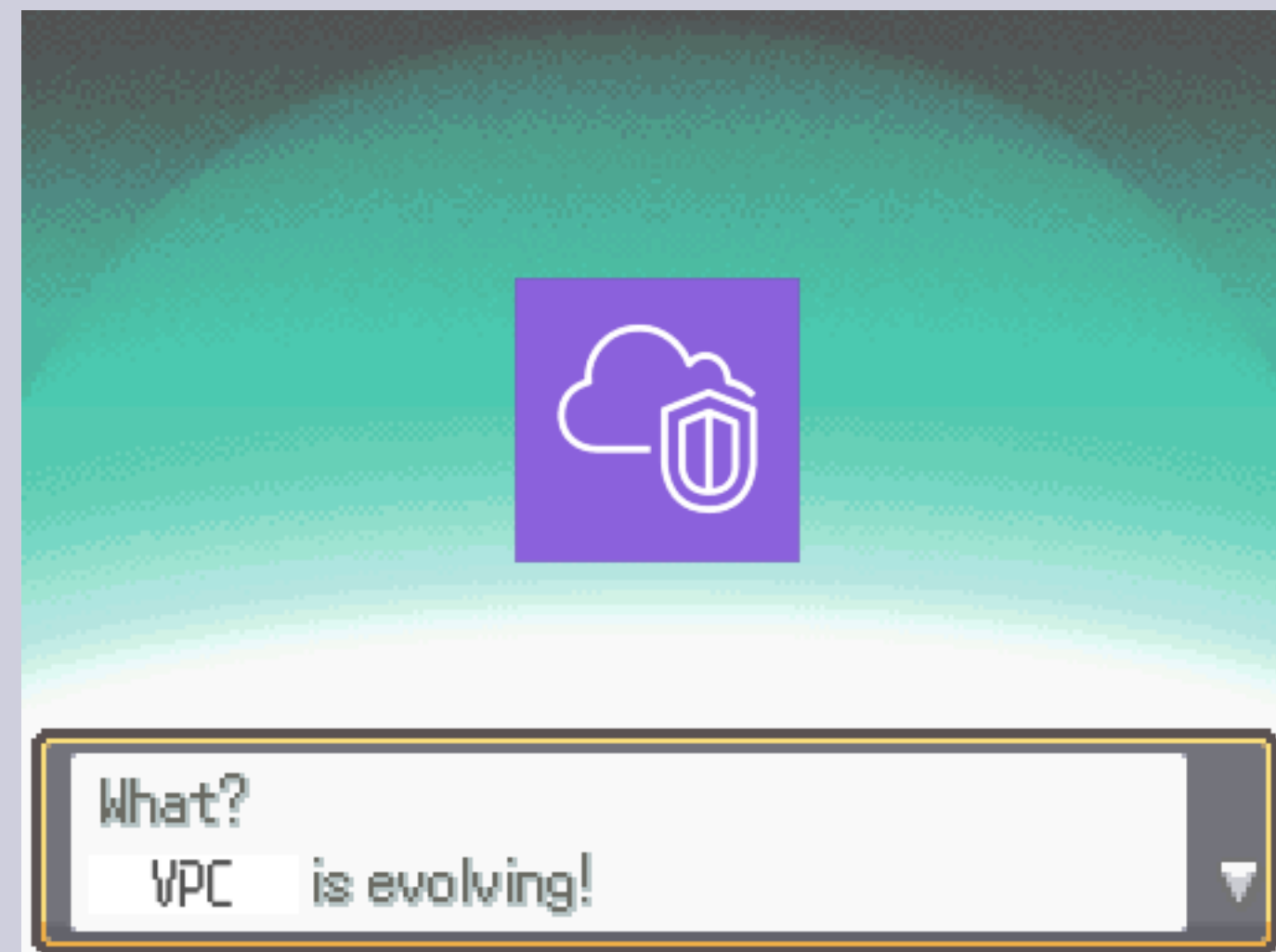
- Molti servizi, come ALB, lavorano cross subnet ma **non cross VPC**
- Necessari troppi **redeploy** e mantenimento di **versioni parallele**

Evolviamo la VPC attuale con i seguenti obiettivi:

- **HA** su tutte subnet private
- Avere solo 6 sottoreti (1 pubblica + 1 privata per AZ)
 - Sottoreti private **più grandi**
 - **Eliminare ambiguità** durante la creazione di nuove risorse

Obiettivo bonus in ottica modernizzazione:

- Aggiunta di supporto a ✨**IPv6** ✨





PLANNING

Measure twice, cut once

PLANNING

2. Vecchia VPC, Nuovo CIDR

È possibile aggiungere **nuovi blocchi CIDR** a una VPC:

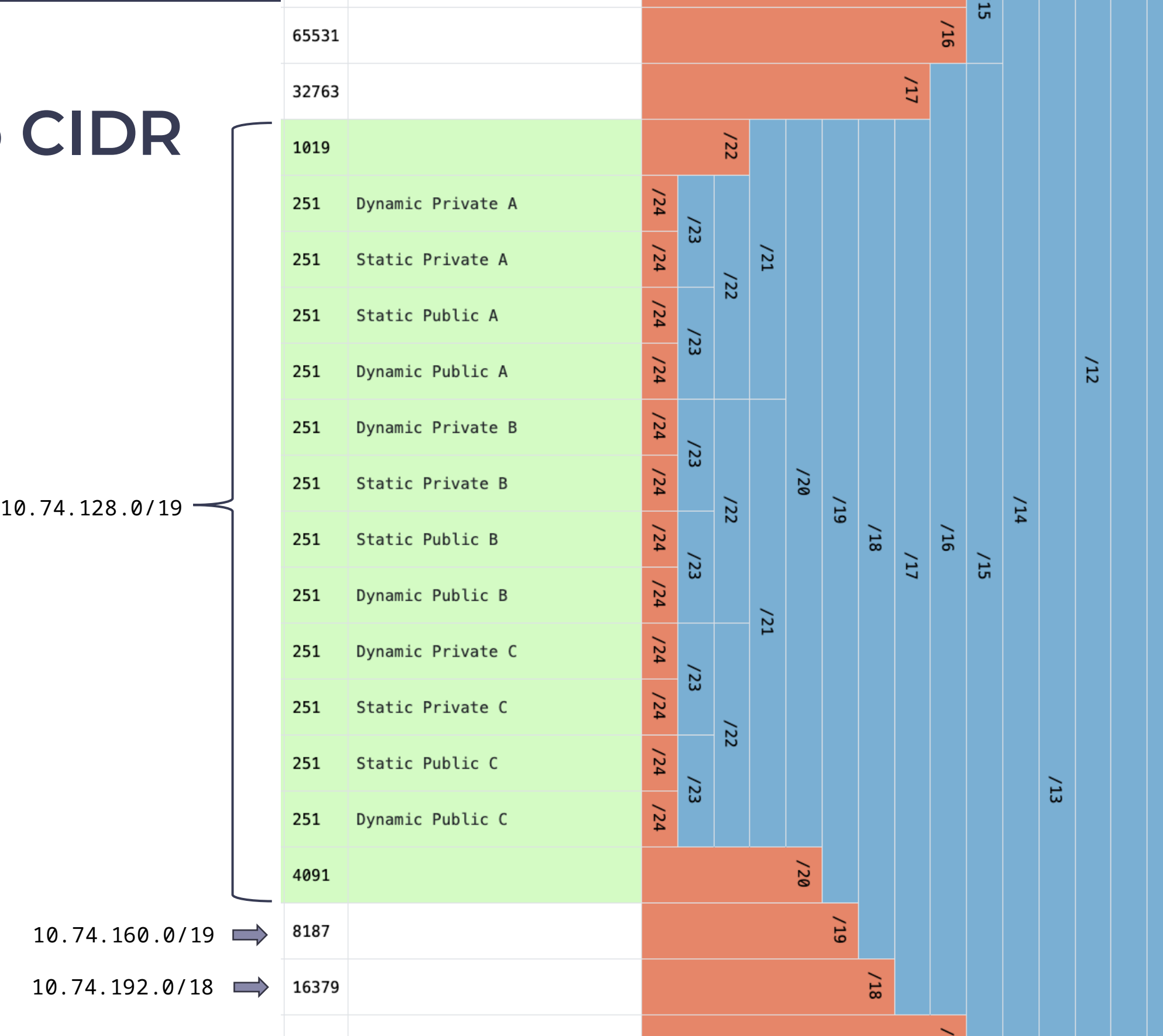
- Non necessariamente adiacenti
- Massimo 4

Vogliamo tenere un **CIDR unico** per pulizia e per compatibilità con MongoDB Atlas

- Prendiamo i due blocchi subito dopo
- Fanno parte dello stesso blocco /17

10.74.128.0/19 > 10.74.128.0/17

Rimane spazio sorpa per espansioni future



PLANNING

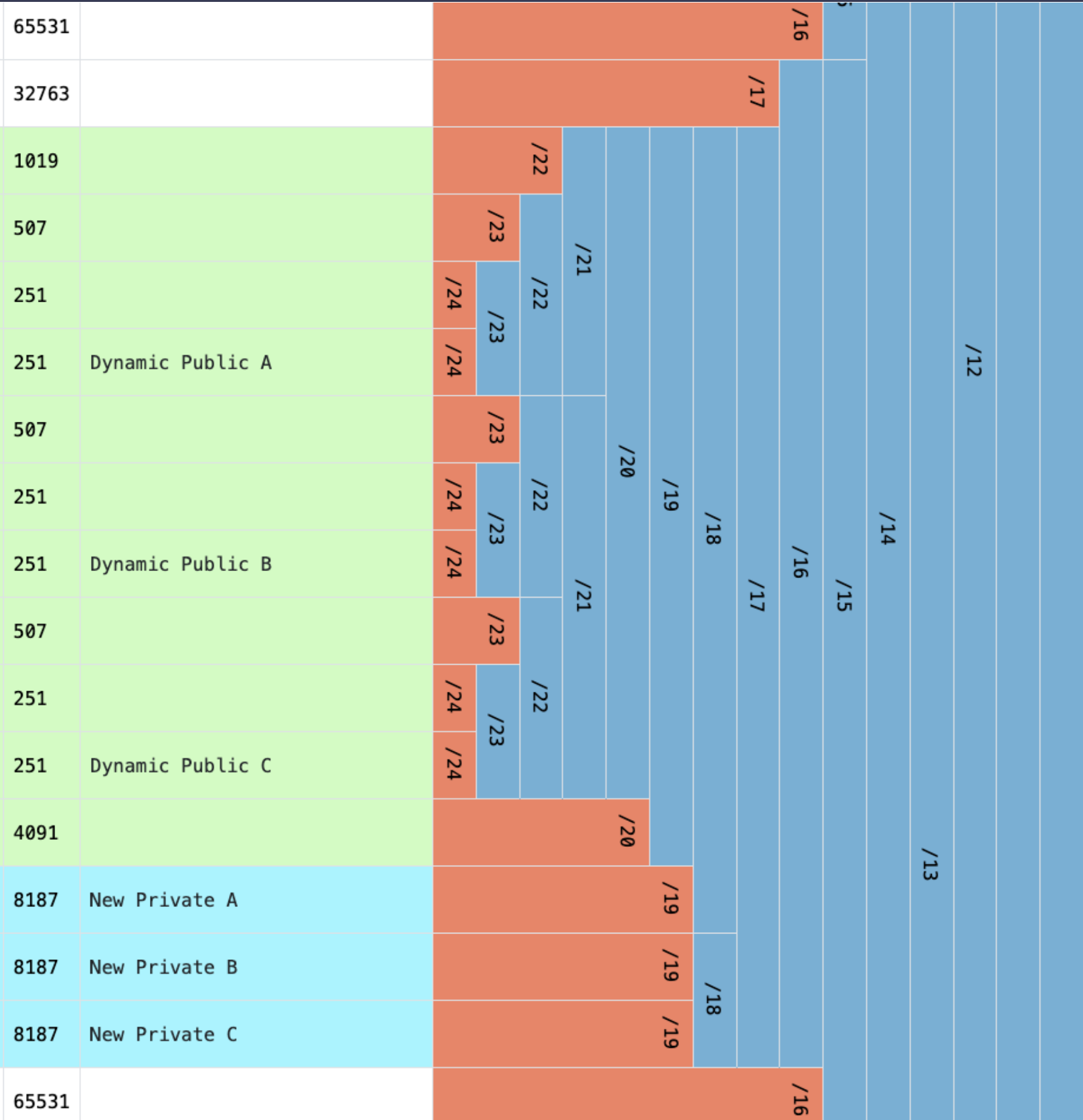
2. Nuove sottoreti

- Usiamo le nuove aggiunte per le nuove sottoreti private:
- Singola sottorete più grande della vecchio VPC
 - Temporanea convivenza con le vecchie sottoreti > migrazione risorse

Cancelliamo un set di sottoreti pubbliche, non ci mancano IP

Cancelliamo tutte le vecchie private

- Rimangono dei buchi nel vecchio blocco
- Non stiamo salvando vite, va bene così
 - Compromesso accettabile



PLANNING

2. IPv6

IPv6 indirizzi di 128 bit (IPv4 32):

- IPv4: ~4,3 miliardi di indirizzi (2^{32})
- IPv6: ~340 undecilioni di indirizzi (2^{128}) 🤖 🤖 🤖

AWS ci fornisce dei blocchi CIDR /56:

- Non possiamo scegliere il blocco come su IPv4
- Comune usare sottoreti /64
- Otteniamo 256 sottoreti da $2^{64} = 1.8 \times 10^{18}$ indirizzi

Route tables, security groups ecc devono avere **doppie entry** IPv4-IPv6.

Assegniamo un CIDR ad ogni sottorete

- 2001:db8:1234:1a00::/56
- ::/0



PLANNING

2. IPv6

Nessuna distinzione fra IP Pubblici e Privati:

- Utilizzo di **EIGW** per tenere le subnet private... private

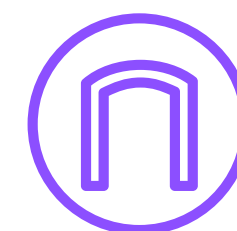
Alcuni bonus:

- Gli indirizzi IPv6 sono **free or charge** ($\neq$ IPv4)
- Traffico per EIGW free of charge ($\neq$ NAT)

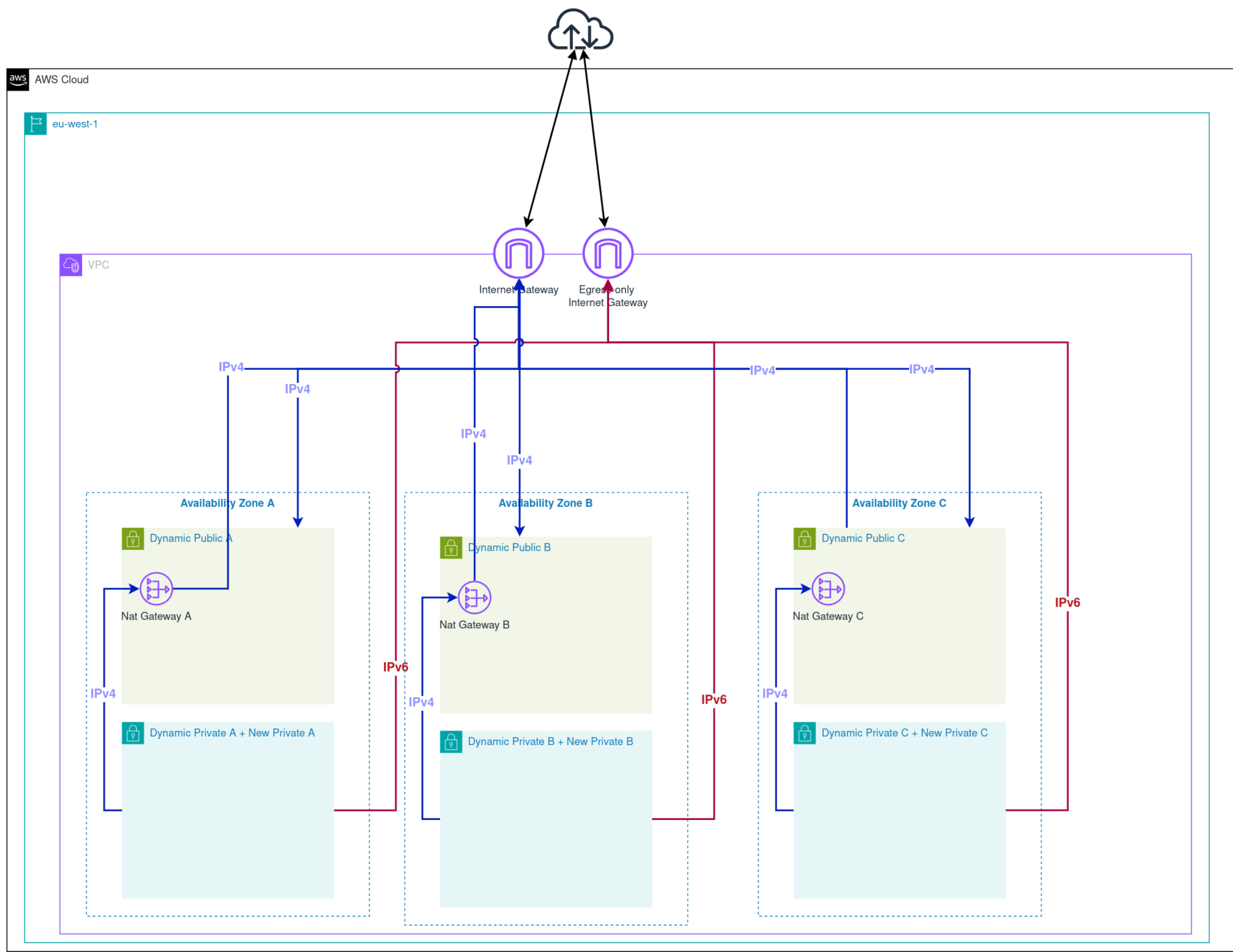
Lambda, EC2, ECS hanno tutti dei flag per abilitarlo o meno:

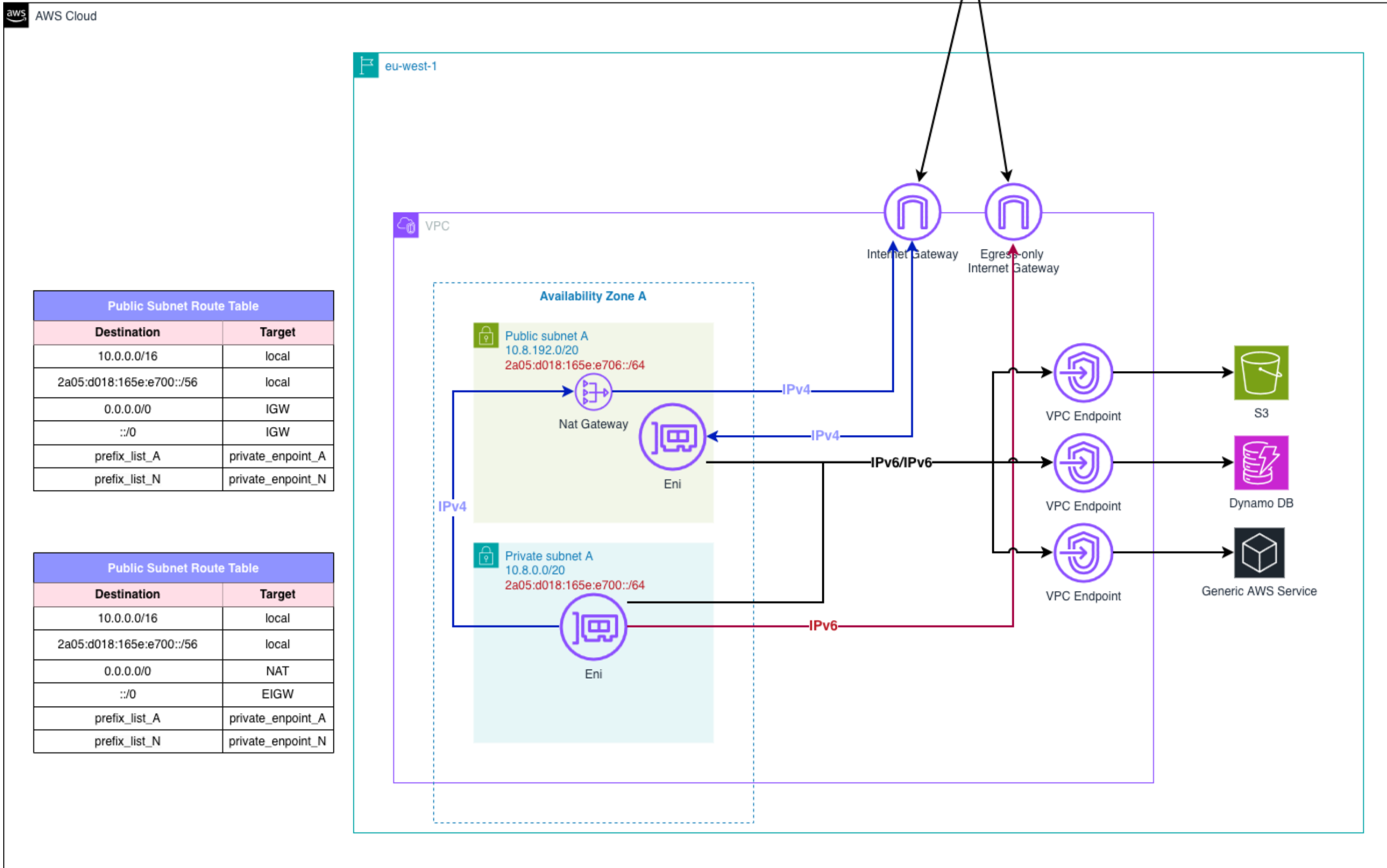
- Se usiamo indirizzi **dualstack**, vengono risolti preferibilmente in IPv6
- Non tutti i servizi AWS possono essere fruiti in IPv6 (per ora)

EGRESS-ONLY INTERNET GATEWAY (EIGW)



Un **Egress Only Internet Gateway** è un componente che permette il traffico IPv6 in uscita verso internet bloccando al contempo le connessioni in entrata.







DALLA TEORIA

Alla pratica

DALLA TEORIA ALLA PRATICA

3. Tooling

Gestiamo tutta la nostra infrastruttura tramite con
approccio IaC

Il nostro tool of choice è **AWS CDK**:

- Codice infrastruttura con linguaggi di programmazione convenzionali
- Layer di astrazione sopra Cloudformation

CDK = Compilatore per Cloudformation +
coordinazione di alcuni deploy

In alcuni casi conviene bypassarlo completamente

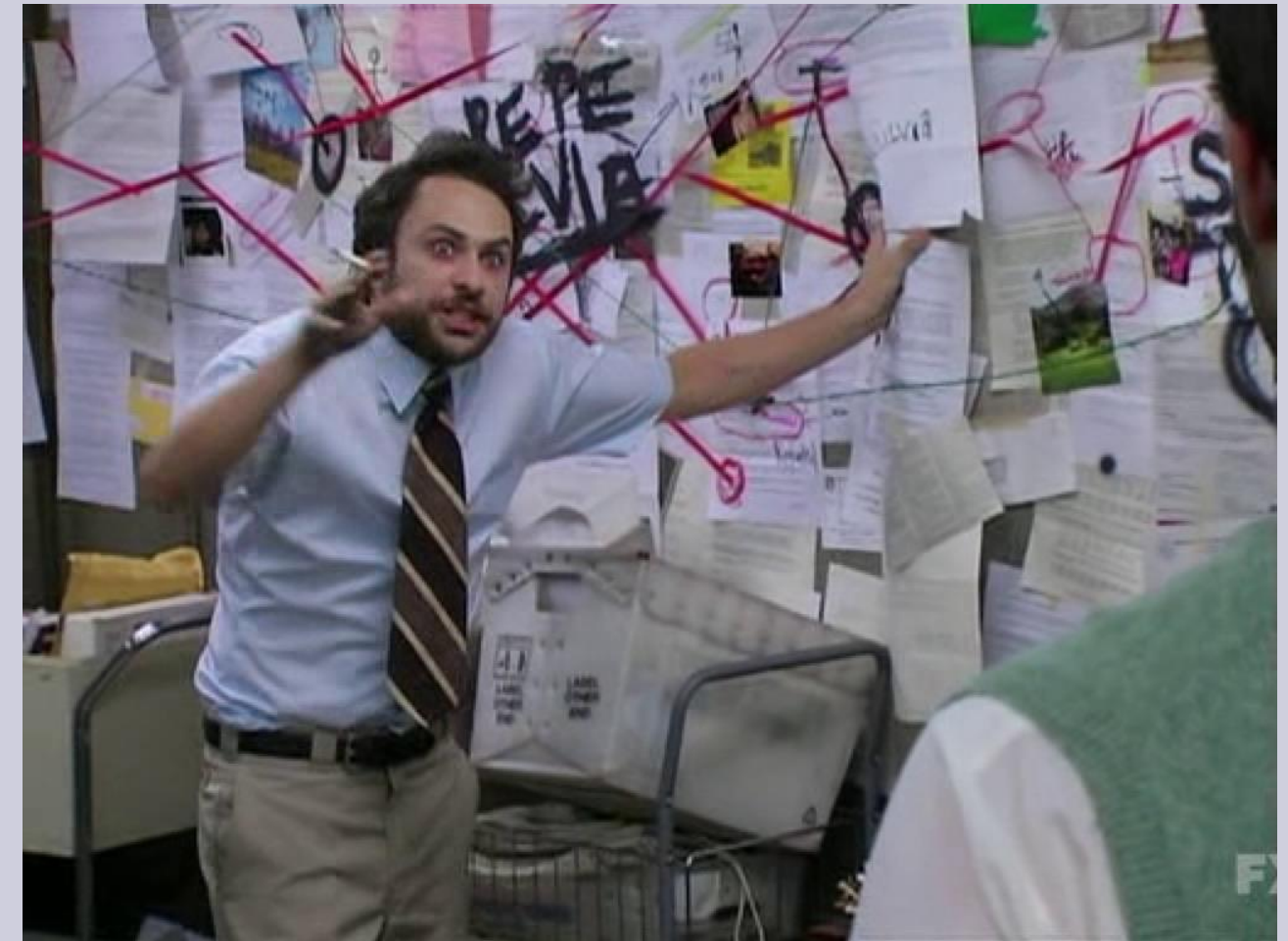


DALLA TEORIA ALLA PRATICA

3. Piano d'azione

Seguiremo in seguenti passaggi:

- Importazione della VPC attuale con CDK
- Evoluzione tramite IaC della VPC
- Aggiornamento delle risorse relative al Network
- Migrazione delle risorse dalle vecchie sottoreti
- Cancellazione delle vecchie sottoreti



DALLA TEORIA ALLA PRATICA

3. Import della VPC

Importare risorse in Cloudformation:

- Scrivere un template che **matcha perfettamente** l'infrastruttura esistente
- Mettere tutte le risorse con **DeletionPolicy: Retain**
- Attributi da API e Cloudformation solitamente sono mappati ~1:1

Occhio a:

- Controllare che tutte le risorse siano importabili (non è sempre stato il caso)
- Matchare bene proprietà e risorse, drift detection non ottimale

Processo tedioso ed error prone, ma non rischioso:

- Procedere a piccoli chunk
- Necessario usare la CLI per alcuni tipi di risorse più nascosti

Identify resources

Resources to import (74)

The following logical IDs are new to your template. Provide the identifier values for the resource you are importing.

RTPublic

AWS::EC2::RouteTable

RouteTableId

Identifier property

RTPriateA

AWS::EC2::RouteTable

RouteTableId

Identifier property

RTPriateB

AWS::EC2::RouteTable

RouteTableId

Identifier property

RTPriateC

AWS::EC2::RouteTable

RouteTableId

Identifier property

EgressOnlyInternetGateway

AWS::EC2::EgressOnlyInternetGateway

Id

Identifier property

DALLA TEORIA ALLA PRATICA

3. Evoluzione VPC

La parte più delicata di tutto il processo:

- Errori possono provocare **down di tutti i servizi**
- Test, test a ancora test in più ambienti

Cloudformation in generale:

- Create > Update > Delete
- Non sempre consistente
- Spesso occorre fare **più passaggi**, ad esempio per rinominare risorse con nome univoco

Nel nostro caso:

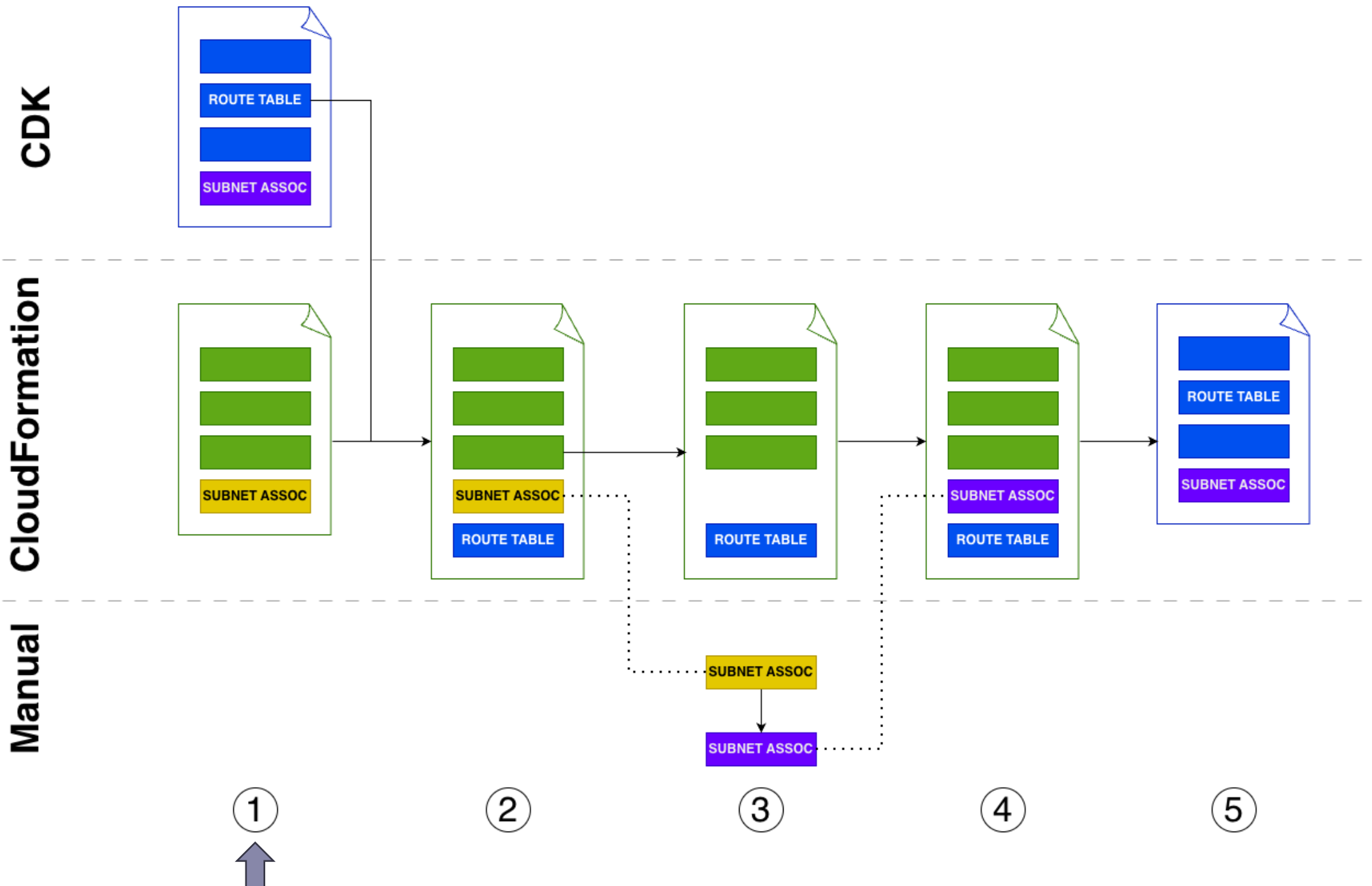
- Alcuni rename da fare in più passaggi
- Procedura a step con commit taggati

L'aggiornamento con Cloudformation di
AWS::EC2::SubnetRouteTableAssociation avrebbe
causato **15 secondi down**



DALLA TEORIA ALLA PRATICA

3. Procedura di aggiornamento



1. Sintesi del template

2. Trapianto di risorse (ex. RouteTable, Nat) nel nuovo template

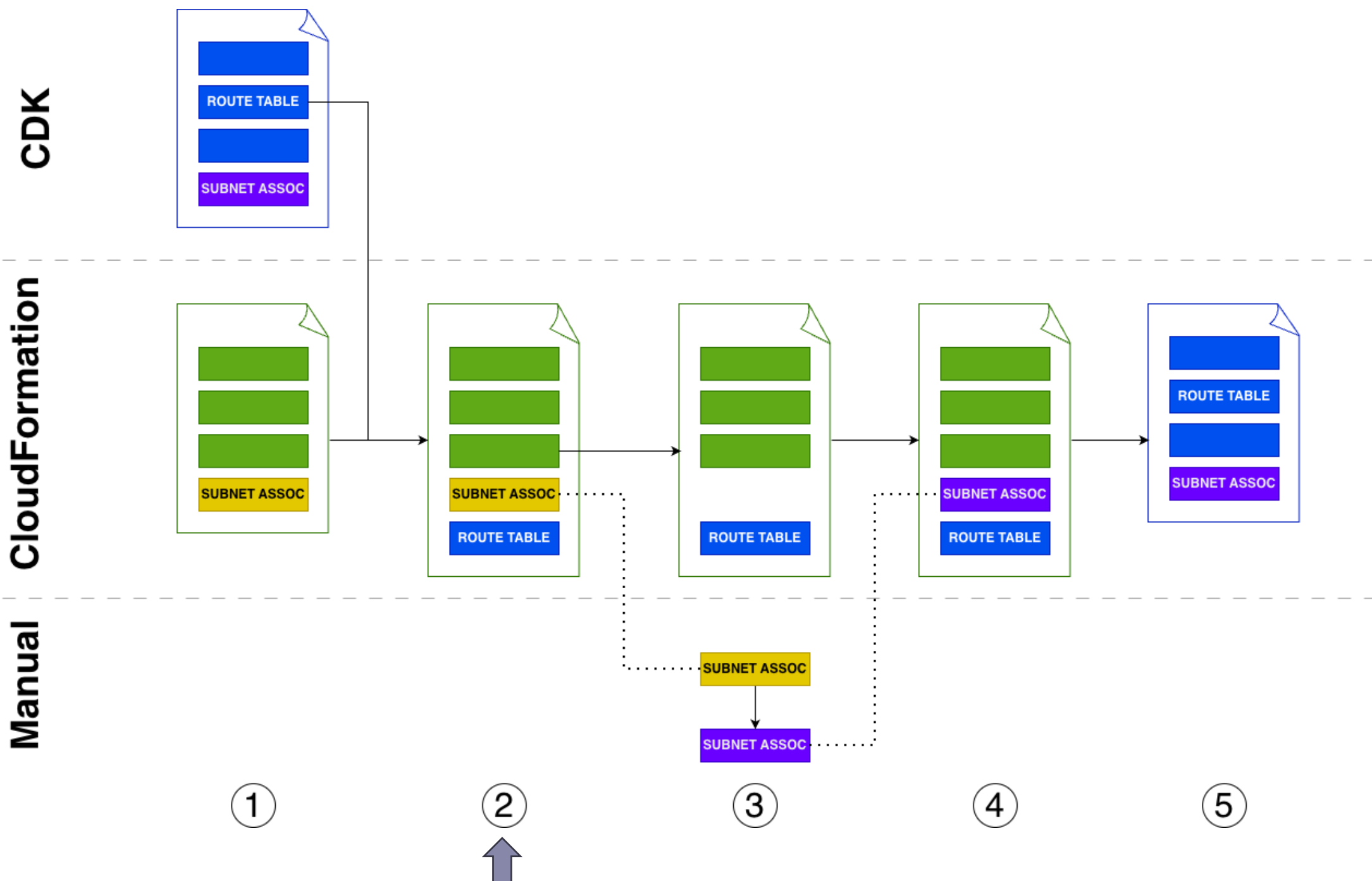
3. Rimozione senza cancellazione della SubnetRouteTableAssoc. Update manuale da console della stessa

4. Rimozione senza cancellazione della SubnetRouteTableAssoc. Update manuale da console della stessa

5. Deploy template con CDK

DALLA TEORIA ALLA PRATICA

3. Procedura di aggiornamento



1. Sintesi del template

2. Trapianto di risorse (ex. RouteTable, Nat) nel nuovo template

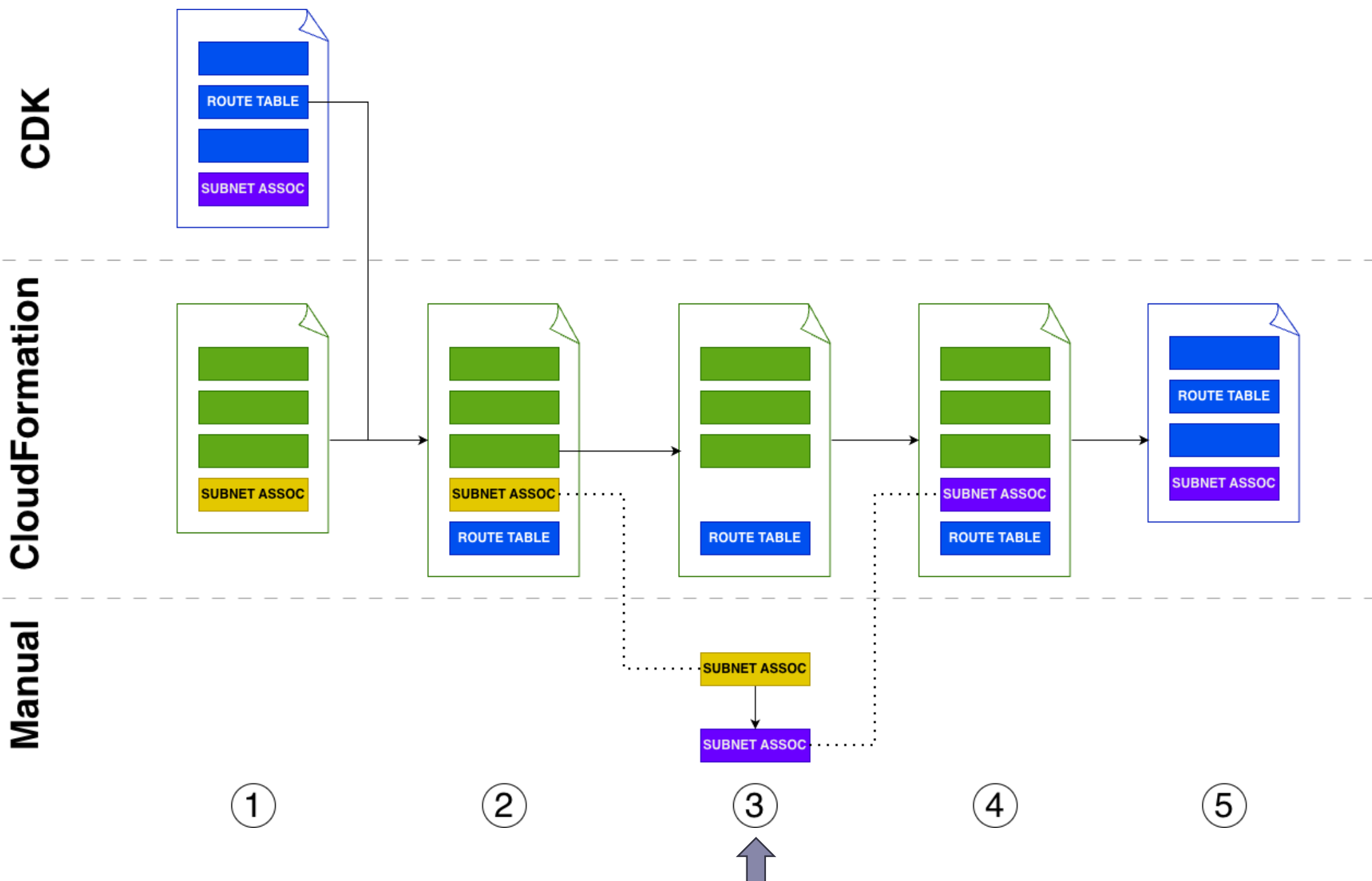
3. Rimozione senza cancellazione della SubnetRouteTableAssoc. Update manuale da console della stessa

4. Rimozione senza cancellazione della SubnetRouteTableAssoc. Update manuale da console della stessa

5. Deploy template con CDK

DALLA TEORIA ALLA PRATICA

3. Procedura di aggiornamento



1. Sintesi del template

2. Trapianto di risorse (ex. RouteTable, Nat) nel nuovo template

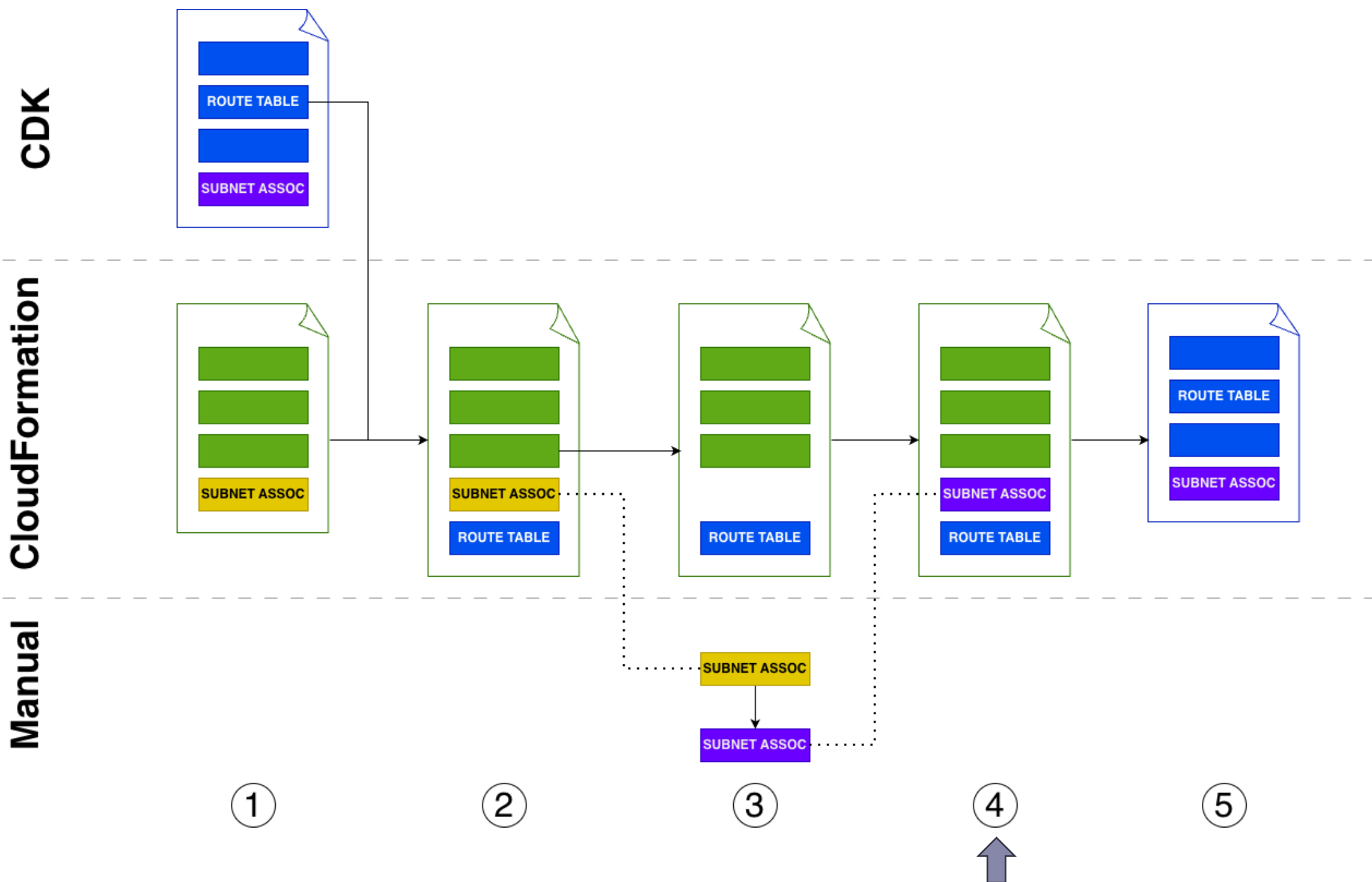
3. Rimozione senza cancellazione della SubnetRouteTableAssoc. Update manuale da console della stessa

4. Rimozione senza cancellazione della SubnetRouteTableAssoc. Update manuale da console della stessa

5. Deploy template con CDK

DALLA TEORIA ALLA PRATICA

3. Procedura di aggiornamento



1. Sintesi del template

2. Trapianto di risorse (ex. RouteTable, Nat) nel nuovo template

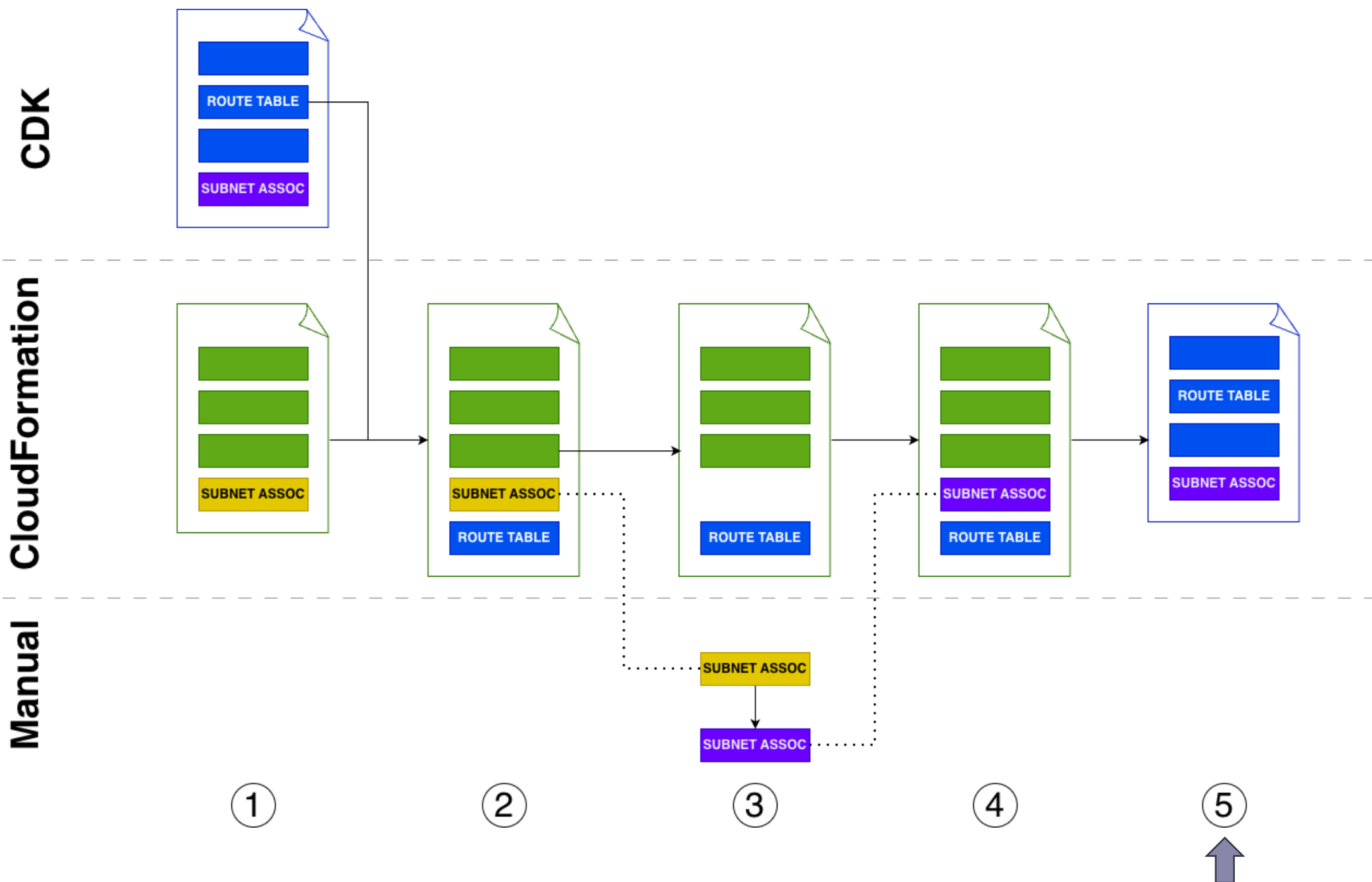
3. Rimozione senza cancellazione della SubnetRouteTableAssoc. Update manuale da console della stessa

4. Rimozione senza cancellazione della SubnetRouteTableAssoc. Update manuale da console della stessa

5. Deploy template con CDK

DALLA TEORIA ALLA PRATICA

3. Procedura di aggiornamento



1. Sintesi del template

2. Trapianto di risorse (ex. RouteTable, Nat) nel nuovo template

3. Rimozione senza cancellazione della SubnetRouteTableAssoc. Update manuale da console della stessa

4. Rimozione senza cancellazione della SubnetRouteTableAssoc. Update manuale da console della stessa

5. Deploy template con CDK

DALLA TEORIA ALLA PRATICA

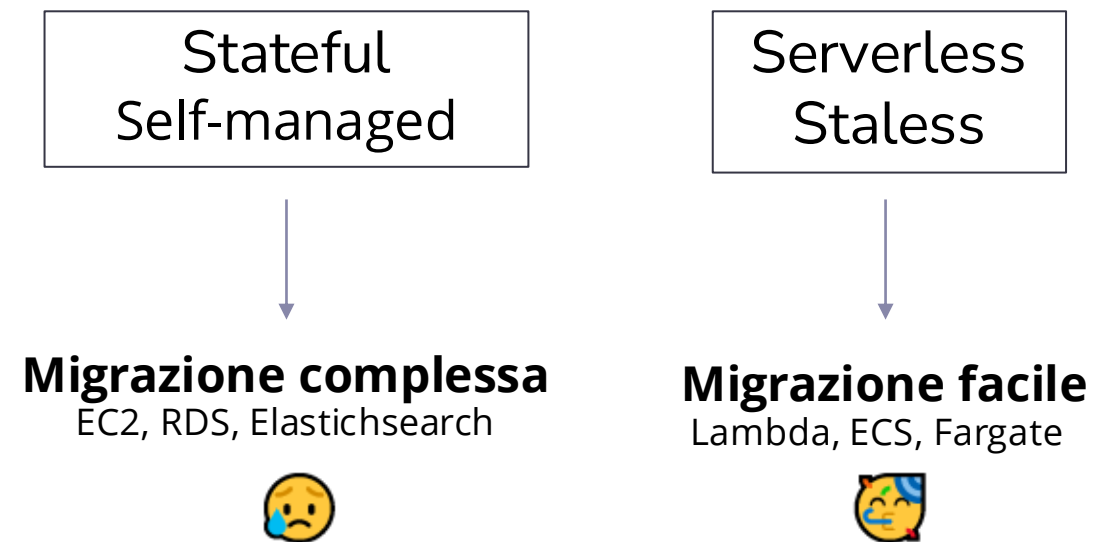
3. Migrazione Risorse

A questo punto avevamo in tutto 15 subnet:

- 6 definitive
- 9 da **svuotare e cancellare**

Scansione di tutte le **ENIs** spostare:

- Circa 260
- Maggior parte lambda 70 Lambda = 210 ENIs



DALLA TEORIA ALLA PRATICA

3. Lambda, Fargate, ECS

Cluster ECS

- Cambio configurazione del cluster e **instance refresh**
- I servizi prendono le sottoreti del cluster

Fargate, Lambda:

- Aggiornamento **libreria CDK** interna a THRON
- Cambio sottoreti di default
- Team di sviluppo bumpano i progetti

DALLA TEORIA ALLA PRATICA

3. Lambda, Fargate, ECS

I **task schedulati** con Fargate non compaiono nella lista ENI

- Creare una regola su Eventbridge che ne logga la partenza, e poi spostarli

Alcune Lambda possono condividere la stessa ENI se stesse subnet e stesso **security group**

- Description della ENI può essere forviante
- Trovabili con query tramite AWS CLI

Alcune Lambda possono avere più **versioni**:

- Cancellare versioni vecchie
- Uso del tool **Lambda ENI Finder** (aws-labs)



DALLA TEORIA ALLA PRATICA

3. Altre risorse

EC2, RDS, Elasticsearch clusters:

- In generale non si possono spostare di subnet
- Ricreare e spostare dati

ALB, NLB, API Gateway APIs, EFS:

- Si possono spostare 1/2 subnet (AZ) alla volta
- Possibile perdita di connessioni nei servizi nella stessa AZ
- Scelti periodi di basso traffico

DALLA TEORIA ALLA PRATICA

3. Note su CDK

CDK aggiunge molta astrazione, ma ad un costo:

- Selezione di **valori di default implicita**. Ad. Esempio selezione di subnet private/pubbliche.
- Costrutti L3 (alto livello) creano mooolte risorse accessorie **opache** e difficili da modificare

Spesso necessari **lungi workaround** con Cloudformation

Occhio al CIDR:

- CDK (ma anche Terraform o altri tool) tornano sempre il **CIDR originale** della VPC
- Necessarie precauzioni a livello di libreria/account



DALLA TEORIA ALLA PRATICA

3. Cancellazione Sottoreti

Aggiunta una **Deny-All NACL** per un paio di settimane

Nessun errore riscontrato:

- Cancellazione tramite deploy CDK

Decisamente lo step più soddisfacente



DALLA TEORIA ALLA PRATICA

4. Key Takeaways

Conosci gli strumenti

Per evitare errori o effetti indesiderati. Essere pronti a usare pattern porco ortodossi

Non fidarti cecamente delle docs

« L'aggiornamento non richiede interruzione » potrebbe essere vero solo dal punto di vista infrastrutturale ma non funzionale

Non assumere che tutti capiscano o si interessino a quello che stai facendo

Alle persone interessa del network solamente quando smette di funzionare. Per avere l'aiuto di tutti è necessario rimuovere al minimo lo sforzo operativo.

Testa, Testa e testa di nuovo

Ho già detto di testare?



INFO

Sebastiano Caccaro

DevOps Engineer@ THRON

 [linkedin.com/in/sebacaccaro/](https://www.linkedin.com/in/sebacaccaro/)

 sebastianocaccaro@gmail.com

 dev.to/sebacaccaro



